

You are completely right. In new developments or massive construction complexes (like a project with 6 separate towers built on one single, massive master Gush/Helka plot), you will be flooded with dozens of distinct building files (תיקי בניין).

Looking back at your Back Office (BO) column layout image, you actually have a dedicated column named תב"ע ומגרש (TABA and Migrash). This is the perfect surgical tool to cut through the noise of multi-building master parcels.

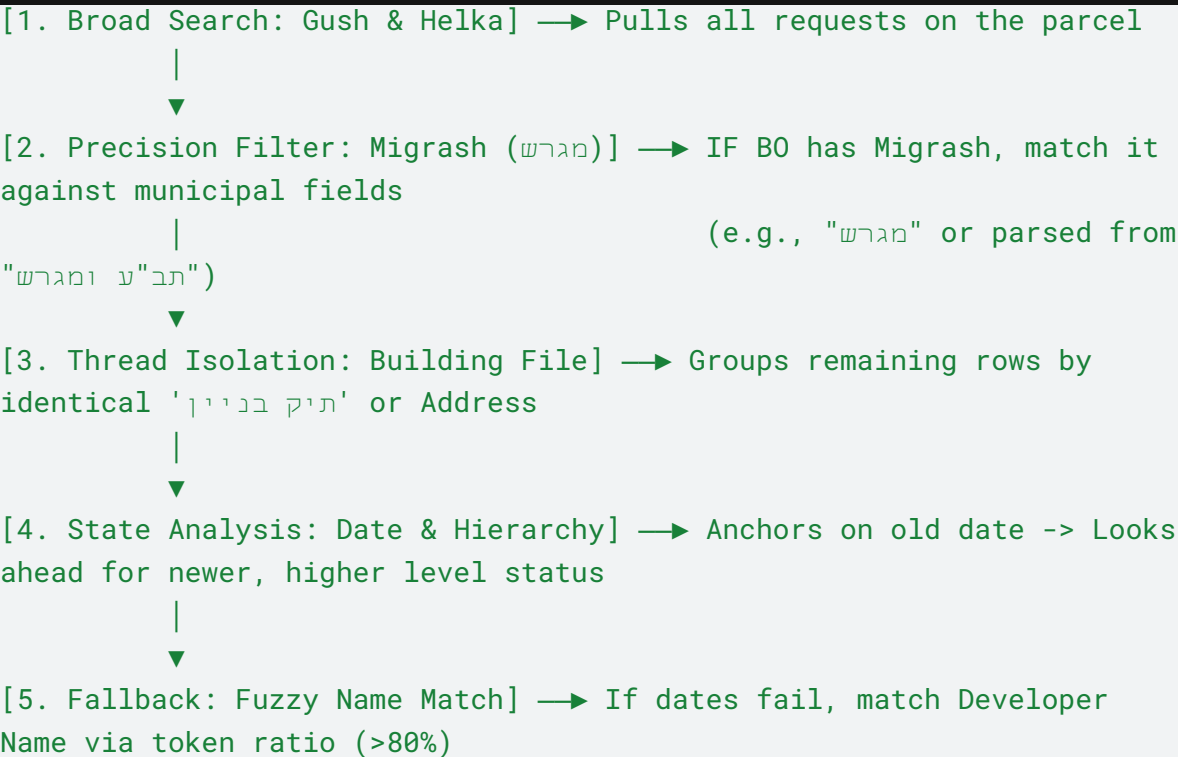
Here is how we integrate the "Migrash Override" into your State Machine hierarchy, followed by your finalized Cline prompt.

The Updated Structural Matching Hierarchy

When the crawler processes a Gush/Helka and receives a large list of results, it should apply filters in this exact order of precision:

Plaintext

None



Phase 1: Data Preparation Updates (Your Tasks)

To make this work, you need to tell Cline how to parse your BO תב"ע ומגרש field. Usually, this field contains something like 5 מגרש/1234/נת or just a number like 5.

You will want to ensure your input data separates or clearly identifies the Migrash number so the Python script can read it cleanly.

Phase 2: Finalized Prompts for Cline

Prompt 1: Building the "Brain" (Hierarchy + Migrash Filter)

Open Cline, ensure it can read your BO Excel structure, and paste this updated prompt:

Context: I am building a "Monitoring Crawler" to track real estate construction milestones across Israeli municipal websites using a State Machine approach based on Date Anchoring and Status Hierarchies.

Task: Create `utils/state_machine.py` and `main.py`. Please add `thefuzz` and `python-dateutil` to my `requirements.txt`.

Requirements for `state_machine.py`:

1. **Status Hierarchy:** Define a dictionary mapping municipal status strings to numeric levels (1=Request, 2=Committee, 3=Permit, 4=Form4).
2. **The Evaluation Logic:** Create a function `find_next_milestone(bo_row_data, scraped_requests_list)`. `bo_row_data` will contain the project's current status level, current status date, developer name, and **Migrash (מגרש)** identifier extracted from the "תב"ע ומגרש" column.
3. **Step A (Migrash Precision Filter):** If the BO data contains a Migrash number, look through the `scraped_requests_list`. If the municipal data provides a Migrash field (or if it can be parsed from their description/block data), filter the list to *only* include requests matching this Migrash. This isolates our specific project in massive new development areas.
4. **Step B (Date Anchor):** Within the filtered list, find the request that matches the project's current BO status date (allow a +/- 4 day buffer). If found, use its **תיק בניין** (Building File) or Address string to isolate the exact building thread.
5. **Step C (Fuzzy Name Fallback):** If Step B fails to find a date anchor, use `thefuzz.partial_ratio` to match the BO Developer Name against the scraped `requestor_name`. If >80% match, use this request to anchor the building thread.
6. **Step D (Look-Ahead Progression):** Once anchored to a building thread, evaluate all subsequent requests in that thread. Find the one with the highest numeric Hierarchy Level that is strictly greater than the current BO level AND has a date newer than the current BO date. Return this new status, date, and request number.

Prompt 2: Building the "Hands" (The Multi-System Scraper Specs)

Once Cline completes the state machine, use this prompt to build the scraper module that pulls the raw data, explicitly making sure it captures the Migrash field if the city webpage displays it.

Context: We are continuing the Monitoring Crawler. We need the scraper component to feed data into our `state_machine.py`.

Task: Create `scrapers/complot_monitor.py`.

Requirements:

1. Create a function `scrape_gush_helka_data(driver, city_url, gush, helka)`.
2. Use `undetected-chromedriver` and explicit `WebDriverWait` selectors.
3. Input the Gush and Helka on the Complot search page and extract *every* result row.
4. For each row, capture: Request Number, Request Type/Status, Date, Address, Requestor Name, Building File (**תיק בניין**), and **Migrash (מגרש)** if it is visible in the results table or expandable details.
5. Standardize all extracted dates to `YYYY-MM-DD` objects.

6. Return the raw list of dictionaries back to `main.py` so it can be evaluated by the state machine logic.

This completely bulletproofs your project tracking. By filtering down by Migrash first, you stop the crawler from accidentally looking at a neighbor's tower data, even if both projects are moving through milestones at the exact same time on a massive shared parcel.

This is an excellent catch. In real estate development, tracking the "pre-planning" or "land banking" phase before any official paperwork is filed is crucial.

Because our current logic relies heavily on **Date Anchoring** (finding a scraped request that matches a known Backoffice date), an empty BO date would instantly break the chain.

We need to introduce a **"Level 0 (Pre-Request Discovery)"** path into the state machine. Here is how we adjust the logic and the prompts to handle projects that don't exist in the municipality's system yet.

The "Level 0" Discovery Logic

When the state machine receives a BO row with no dates (Level 0), it skips the Date Anchor entirely and relies on a combination of **Location (Migrash)** and **Identity (Developer Name)**.

1. **The Broad Query:** Search Gush/Helka.
2. **The Precision Filter:** Filter by Migrash (if BO provides one).
3. **The Identity Anchor:** Since we have no dates to anchor to, we evaluate all remaining requests using `thefuzz.partial_ratio`. We compare the BO Developer Name to the scraped Requestor Name.
4. **The Baseline Capture:** If we find a request with an 80%+ name match on that exact parcel/migrash, we capture it. This is our "Discovery." The project has officially moved from Level 0 to Level 1 (or higher, if they progressed fast).

Phase 1: Data Preparation Updates (Your Tasks)

Simply add "Level 0" to your Status Hierarchy map.

Level	BO Status Name	Known Municipal Synonyms

0	Pre-Request (Empty)	(N/A - This triggers the discovery logic)
1	Request	הגשת בקשה, בקשה שנקלטה, תאריך קליטה

2	Committee	החלטת ועדה, אישור בתנאים, אושר בתנאים
...		

Phase 2: Finalized Prompts for Cline

We only need to update the prompt for `state_machine.py` (The Brain). The scraper prompt (The Hands) remains exactly the same, as it's still just gathering the raw data.

Updated Prompt 1: Building the "Brain" (Handling Level 0)

Copy and paste this updated prompt into Cline:

Context: I am building a "Monitoring Crawler" to track real estate construction milestones across Israeli municipal websites using a State Machine approach. I need to handle "Pre-Request" projects where the current Backoffice date is empty.

Task: Create `utils/state_machine.py` and `main.py`. Add `thefuzz` and `python-dateutil` to `requirements.txt`.

Requirements for `state_machine.py`:

1. **Status Hierarchy:** Define a dictionary mapping municipal status strings to numeric levels (Level 0 = Pre-Request/Empty, 1=Request, 2=Committee, 3=Permit, 4=Form4).

2. **Evaluation Logic:** Create `find_next_milestone(bo_row_data, scraped_requests_list)`. `bo_row_data` contains `bo_current_level` (can be 0), `bo_current_date` (can be None), `bo_developer_name`, and `bo_migrash`.
3. **Step A (Migrash Precision Filter):** Filter `scraped_requests_list` to only include requests matching `bo_migrash` (if a migrash is provided in the BO data).
4. **Step B (Level 0 Discovery Path):** IF `bo_current_level == 0` or `bo_current_date` is None: Skip date anchoring. Iterate through the filtered list and use `thefuzz.partial_ratio` to compare `bo_developer_name` against the scraped `requestor_name`. If >80% match is found, return the status, date, and request number of the most advanced request in that cluster. **EXIT FUNCTION.**
5. **Step C (Standard Date Anchor):** IF `bo_current_level > 0`: Find the request that matches `bo_current_date` (allow +/- 4 day buffer). If found, use its `תיק בניין` (Building File) or exact Address to isolate the building thread.
6. **Step D (Fuzzy Name Fallback):** If Step C fails, use the `thefuzz` >80% match on the developer name to anchor the building thread.
7. **Step E (Look-Ahead Progression):** Once anchored via Step C or D, evaluate all subsequent requests in that building thread. Find the one with the highest numeric Hierarchy Level strictly greater than `bo_current_level` AND with a date newer than `bo_current_date`. Return this new status, date, and request number.

Requirements for `main.py`:

1. Set up `pandas` to read the BO Excel file. Map empty dates/statuses to Level 0 before passing to the logic.
2. Build the iteration loop to pass data to the scrapers.

By explicitly separating the **Level 0 Discovery Path** from the **Standard Date Anchor**, the crawler will gracefully discover brand-new projects on their very first day of municipal filing, while strictly verifying it's the correct developer on the correct plot.